

[Event dashboard](#) > [AgentCore Runtime](#) > Advanced Concepts for Agentcore Runtime

Advanced Concepts for Agentcore Runtime

Overview

This section explores advanced capabilities of Amazon Bedrock AgentCore Runtime, including streaming responses, session management, large payload handling, and multi-modal content processing. These tutorials demonstrate how to build sophisticated, production-ready agents that can handle complex real-world scenarios with optimal performance and user experience.

Streaming Responses with AgentCore Runtime

Streaming responses allow a responsive user experiences for operations generating large content or requiring significant processing time.

When implementing streaming with AgentCore Runtime:

- **Use** `async def` for your endpoint function
- **Use** `yield` to stream chunks as they become available
- **AgentCore SDK automatically handles** Server-Sent Events (SSE) format
- **Clients receive** `Content-Type: text/event-stream` responses

Example Streaming Implementation

```
1  from bedrock_agentcore.runtime import BedrockAgentCoreApp
2  from strands import Agent, tool
3  from strands.models import BedrockModel
4
5  app = BedrockAgentCoreApp()
6
7  # Configure your agent with tools
8  @tool
9  def get_weather():
10     """Get current weather"""
11     return "sunny"
12
```



```

13 model = BedrockModel(model_id="us.anthropic.claude-sonnet-4-20250514-v1:0")
14 agent = Agent(model=model, tools=[get_weather])
15
16 @app.entrypoint
17 async def streaming_agent(payload):
18     """
19     Streaming entrypoint - yields chunks as they become available
20     """
21     user_input = payload.get("prompt")
22
23     try:
24         # Stream each chunk using async generator
25         async for event in agent.stream_async(user_input):
26             if "data" in event:
27                 yield event["data"]
28     except Exception as e:
29         # Handle errors in streaming context
30         yield {"error": str(e), "type": "stream_error"}

```

Configure Runtime Deployment

We use our [starter toolkit](#) to configure the AgentCore Runtime deployment with an entrypoint. We also configure the starter kit to auto create the Amazon ECR repository on launch

```

1 from bedrock_agentcore_starter_toolkit import Runtime
2
3 agentcore_runtime = Runtime()
4
5 response = agentcore_runtime.configure(
6     entrypoint="streaming_agent.py",
7     execution_role=your_iam_role_arn,
8     auto_create_ecr=True,
9     requirements_file="requirements.txt"
10 )

```



Launch to AgentCore Runtime

Next, we launch the streaming agent to the AgentCore Runtime. This will create the Amazon ECR repository and the AgentCore Runtime

```

1 # Deploy the streaming agent
2 launch_result = agentcore_runtime.launch()

```



Invoke AgentCore Runtime using AgentCore SDK

We invoke the AgentCore Runtime with a payload and receive streaming responses

```
1 # Simple invocation with automatic streaming handling
2 invoke_response = agentcore_runtime.invoke({
3     "prompt": "What's the weather like?"
4 })
```



Alternate Approach - Invoke AgentCore Runtime using boto3 for Advanced Control

Alternatively we can invoke [AgentCore Runtime](#) with any AWS SDK. Here we demonstrate invocation using Boto3.

```
1 import boto3
2 import json
3
4 agentcore_client = boto3.client('bedrock-agentcore')
5
6 response = agentcore_client.invoke_agent_runtime(
7     agentRuntimeArn=agent_arn,
8     qualifier="DEFAULT",
9     payload=json.dumps({"prompt": "Calculate something complex"}))
10 )
11
12 # Handle streaming response
13 if "text/event-stream" in response.get("contentType", ""):
14     for line in response["response"].iter_lines():
15         if line and line.startswith(b"data: "):
16             chunk = line[6:].decode("utf-8")
17             print(f"Received: {chunk}")
```



Session and Context Management

Amazon Bedrock AgentCore Runtime provides isolated sessions for each user interaction, enabling agents to maintain context and state across multiple invocations while ensuring complete security isolation between different users. This example showcases how AgentCore Runtime handles sessions, maintains context across multiple invocations, and how agents can access runtime information through the context object.

For demonstration purposes, we will use a Strands Agent that showcases these session management capabilities.

Session Isolation and Security

AgentCore Runtime provides **complete session isolation** through dedicated microVMs:

- **Dedicated Resources:** Each session runs in its own microVM with isolated CPU, memory, and filesystem
- **Security Boundaries:** Complete separation between user sessions prevents data contamination
- **Deterministic Cleanup:** After session completion, the microVM is terminated and memory is sanitized

Session Lifecycle

Sessions in AgentCore Runtime follow a specific lifecycle:

1. **Creation:** A new session is created on first invocation with a unique `runtimeSessionId`
2. **Active State:** Session processes requests and maintains context
3. **Idle State:** Session waits for next invocation while preserving context
4. **Termination:** Session ends due to:
 - Inactivity (15 minutes)
 - Maximum lifetime (8 hours)
 - Health check failures

Context Persistence

Within a session, AgentCore Runtime maintains:

- **Conversation History:** Previous interactions and responses
- **Application State:** Variables and objects created during execution
- **File System:** Any files created or modified during the session
- **Environment Variables:** Custom settings and configurations

Implementation of Session and Context Management



```
1  from strands import Agent, tool
2  from strands_tools import calculator # Import the calculator tool
3  import argparse
4  import json
5  from bedrock_agentcore.runtime import BedrockAgentCoreApp
6  from strands.models import BedrockModel
7  import asyncio
8  from datetime import datetime
9
10 app = BedrockAgentCoreApp()
11
12 # Create a custom tool
13 @tool
14 def weather():
15     """ Get weather """ # Dummy implementation
16     return "sunny"
17
18 @tool
19 def get_time():
20     """ Get current time """
21     return datetime.now().strftime("%Y-%m-%d %H:%M:%S")
22
23 model_id = "us.anthropic.claude-sonnet-4-20250514-v1:0"
24 model = BedrockModel(
25     model_id=model_id,
26 )
27 agent = Agent(
28     model=model,
29     tools=[
30         calculator, weather, get_time
31     ],
32     system_prompt="""
33     You're a helpful assistant. You can do simple math calculations,
34     tell the weather, and provide the current time.
35     Always start by acknowledging the user's name
36     """
37 )
38
39 def get_user_name(user_id):
40     users = {
41         "1": "Maira",
42         "2": "Mani",
43         "3": "Mark",
44         "4": "Ishan",
45         "5": "Dhawal"
46     }
47     return users[user_id]
48
49 @app.entrypoint
50 def strands_agent_bedrock_handling_context(payload, context):
51     """
```

```

52     AgentCore Runtime entrypoint that demonstrates context handling and sess
53
54     Args:
55         payload: The input payload containing user data and request informat
56         context: The runtime context object containing session and execution
57
58     Returns:
59         str: The agent's response incorporating context information
60     """
61     user_input = payload.get("prompt")
62     user_id = payload.get("user_id")
63     user_name = get_user_name(user_id)
64
65     # Access runtime context information
66     print("=== Runtime Context Information ===")
67     print("User id:", user_id)
68     print("User Name:", user_name)
69     print("User input:", user_input)
70     print("Runtime Session ID:", context.session_id)
71     print("Context Object Type:", type(context))
72     print("=== End Context Information ===")
73
74     # Create a personalized prompt that includes context information
75     prompt = f"""My name is {user_name}. Here is my request: {user_input}
76
77     Additional context: This is session {context.session_id}.
78     Please acknowledge my name and provide assistance."""
79
80     response = agent(prompt)
81     return response.message['content'][0]['text']
82
83 if __name__ == "__main__":
84     app.run()

```

Accessing Context using Context Object Structure

The `context` parameter in your entrypoint function provides access to runtime information:

```

1  @app.entrypoint
2  def strands_agent_bedrock_handling_context(payload, context):
3      # Access session information
4      session_id = context.session_id
5      # Use context information in your agent logic

```



Payload Flexibility

The `payload` parameter allows flexible data passing:

```
1  # Example payload structures
2  payload = {
3      "prompt": "User's question",
4      "user_id": "1",
5      "preferences": {...},
6      "context_data": {...}
7  }
```



This enables rich, structured communication between clients and agents while maintaining the session context provided by the runtime.

Handling Large Multi-Modal Payloads

Amazon Bedrock AgentCore Runtime handles large payloads up to 100MB, including multi-modal content such as Excel files and images.

Why is this important?

Handling Large Multi-Modal Payloads is an important capability for Amazon Bedrock AgentCore Runtime because it enables agents to process and analyze complex, real-world data at scale. In modern business environments, data often comes in various formats and sizes, including large Excel files, high-resolution images, and other rich media. By supporting payloads up to 100MB and providing multi-modal processing capabilities, the AgentCore Runtime allows agents to extract insights from these diverse data sources. This is crucial for empowering agents to tackle sophisticated use cases, make informed decisions, and provide comprehensive analysis – ultimately delivering more value to end-users

Implementation of Handling Large Multi-Modal Payloads

Below code demonstrates how to create an entrypoint for your agent that can process both Excel files and images from large payloads:

```
1  @app.entrypoint
2  def multimodal_data_processor(payload, context):
3      """
4      Process large multi-modal payloads containing Excel data and images.
5  """
```



```

6      Args:
7          payload: Contains prompt, excel_data (base64), image_data (base64)
8          context: Runtime context information
9
10     Returns:
11         str: Analysis results from both data sources
12     """
13     prompt = payload.get("prompt", "Analyze the provided data.")
14     excel_data = payload.get("excel_data", "")
15     image_data = payload.get("image_data", "")
16
17     print(f"=== Large Payload Processing ===")
18     print(f"Session ID: {context.session_id}")
19
20     if excel_data:
21         print(f"Excel data size: {len(excel_data) / 1024 / 1024:.2f} MB")
22     if image_data:
23         print(f"Image data size: {len(image_data) / 1024 / 1024:.2f} MB")
24     print(f"Excel data {excel_data}")
25     print(f"Image data {image_data}")
26     print(f"=== Processing Started ===")
27     # Decode base64 to bytes
28     excel_bytes = base64.b64decode(excel_data)
29     # Decode base64 to bytes
30     image_bytes = base64.b64decode(image_data)
31
32     # Enhanced prompt with data context
33     enhanced_prompt = f"""{prompt}
34     Please analyze both data sources and provide insights.
35     """
36
37     response = agent(
38         [{
39             "document": {
40                 "format": "xlsx",
41                 "name": "excel_data",
42                 "source": {
43                     "bytes": excel_bytes
44                 }
45             },
46         },
47         {
48             "image": {
49                 "format": "png",
50                 "source": {
51                     "bytes": image_bytes
52                 }
53             },
54         },
55         {
56             "text": enhanced_prompt
57         }
58     ]

```



```
58     )
59     return response.message['content'][0]['text']
60
61 if __name__ == "__main__":
62     app.run()
```

Try it out

At an AWS Event

If you are following the workshop via workshop studio, now go to JupyterLab in SageMaker Studio. In the JupyterLab UI navigate to 01-AgentCore-runtime/03-advanced-concepts

Self-paced

Here are notebooks that let you try out the implementations:

Framework	Use Case	Description	Notebook
Strands Agent	Streaming Responses	Implementing streaming responses using Amazon Bedrock AgentCore Runtime with real-time capabilities	streaming-agent-response ↗
Strands Agent	Understanding Runtime Context	Working with runtime context and session management, maintaining context across multiple invocations	understanding-runtime-context ↗
Strands Agent	Handling Multi-Modal Payloads	Processing large payloads up to 100MB including multi-modal content such as Excel files and images	handling-large-payloads ↗

[Previous](#)

[Next](#)

© 2008 - 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

[Privacy policy](#)

[Terms of use](#)

[Cookie preferences](#)